

Defense Speech — 19 slides

Intelligent Similarity Search System for the Gemba Walk Assistant

Mohamed Mortadha Agoubi · iObeya · 2025/2026

Updated for the new deck: Architecture, Built with, The pipeline, RAG · Retrieval and RAG · Generation were added. The "Testing changed the architecture" slide was removed — that story now lives in the Q&A section, because it is still your best answer to "what was difficult?".

⚠ Read this first — the deck is now 20.8 minutes

The slide registry budgets **20.8 minutes**. Your limit is **15**.
So this speech is written *shorter than the slides suggest*, and you have a choice to make **before** you walk in.

Route	What you do	Speech	Slack inside 15 min
A+ — recommended	skip slides 11 & 12, and drop the ⌚ lines	12 min 30	2 min 30
A	skip slides 11 & 12	13 min 00	2 min 00
B	everything, every line	13 min 50	1 min 10

Route A — skip slide 11 (Built with) and slide 12 (The pipeline)

Press **0**, click slide 13, keep going. Nobody notices a skipped slide.
Why those two: slide 10 already shows the stack inside the architecture, and slides 13 to 15 already explain the pipeline in more useful detail.
You lose repetition, not content.

Route B — all 19 slides

Only if you have rehearsed twice and the jury is relaxed about time.
It fits inside 15 minutes *only if you do not hesitate*.

🛑 **If you are lost on time mid-defense:** jump straight to slide 17 (Results). Results → Conclusion → Thank you is a complete, dignified ending.

How to use this document

Short lines, one idea per line. Do not memorise the words.
Memorise the **order of the ideas**. The words will come.

Symbol	Meaning
▶ CLICK	press the right arrow — the slide advances one step
(pause)	stop for one second. It is not silence, it is punctuation.
⌚	a line you can cut if you are running late
✂	a whole slide you can skip (Route A)
🛑	your safety net — if you lose your words, say this one sentence , then move on

The three rules that save you

1. **Speak slowly.** Slower than feels natural. You are judged on *clarity*, never speed.
2. **If you lose a word, do not go back.** Say the 🛑 line and move to the next slide. Nobody in the room knows what you planned to say.
3. **Every slide has one job.** Remember the job, improvise the rest.

Words to practise out loud

Word	Say it like	Note
Gemba	GEM -ba	hard G, like "get"
Obeya / iObeya	o- BAY -ya	
semantic	se- MAN -tic	
meaning	MEE -ning	you will say it ~10 times
retrieval	ri- TREE -val	or just say " finding "
threshold	THRESH -old	two parts
precision / recall	pre- SI -zhun / ri- CALL	
embedding	em- BED -ding	or say " a list of numbers "
coverage	CUH -ve-ridge	
microservice	MY -cro-service	
architecture	AR -ki-tec-ture	

If a word feels dangerous, replace it. "retrieval" → "finding". "embedding" → "a list of numbers". Simple English is professional English.

THE SPEECH

Slide 1 — Cover · ~40 s

(Stand still. Look at the jury. Smile. Then start.)

Good morning everyone.

Thank you for being here today.

My name is Mohamed Mortadha Agoubi.

I did my End-of-Studies Project at iObeya.

(pause)

My subject is an Intelligent Similarity Search System
for the Gemba Walk Assistant.

In simple words: I taught a factory tool to **remember**.

To remember every problem the company has already solved.

(pause)

Let me show you why this matters.



I taught a factory tool to remember the problems the company already solved.

Slide 2 — Agenda · ~28 s · 3 clicks

Here is where we are going together.

► **CLICK**

First the context. Then the problem — and it is a serious one.

Then the solution, and I will show it running.

Then what makes it different from a normal search.

And finally, the results.

► **CLICK**

One idea holds all of this together.

Every problem a company solves is an **asset**.

Today, that asset is filed away — and forgotten.



Context, problem, solution, difference, results.

Slide 3 — Lean · ~33 s · 3 clicks

Let me start with Lean.

Lean is a way of working.

Create more value with fewer resources.

Remove waste, and solve problems where they happen.

► **CLICK** *(the five principles — do not read them)* 🕒

► **CLICK**

I only need three words from this world.

Gemba — it means "the real place". The shop floor.

Gemba Walk — you go there, and you see the problem with your own eyes.

And **continuous improvement** — every problem you solve becomes a standard.

(pause)

Keep that last one in mind. It is the heart of my project.



Gemba means the real place. A Gemba Walk is going to see the problem yourself.

Slide 4 — Obeya → iObeya · ~38 s · 4 clicks

So where does iObeya come from?

"Obeya" is a Japanese word. It means "the big room".

At Toyota, they put the whole project on the walls of one room.

Everyone comes in, sees the same thing, and decides together.

(point at the photos) *For years, this was paper. On walls. In one building.*

► **CLICK**

iObeya made it digital. Same rituals. Same boards. No walls.

► **CLICK**

Today it is used by Sanofi, Air France, Renault, Airbus, Thales, Volvo.

Factories where one problem costs real money.



iObeya is the digital version of the Toyota "big room".

Slide 5 — The Gemba Walk Assistant · ~48 s · 4 clicks

Inside iObeya there is one application for the shop floor.

It is called the **Gemba Walk Assistant**.

A manager walks the floor. He sees a problem. He writes it down.

But not as free text.

► **CLICK**

He fills a standard Lean form — the **5W2H**.

What happened. Who. Where. When. How. How many. And why it matters.

► **CLICK**

Then the team works on it: contain it, find the root cause, apply a solution, write the new standard.

► **CLICK**

(slow down — this is the key sentence of the first part)

And here is the important part.

When the problem is solved, the issue **stays** on the board.

With its root cause. Its solution. And the actions that worked.

(pause)

So the company is already storing all the answers.

 **Every solved problem stays on the board, with its solution attached.**

Slide 6 — The problem · ~52 s · 3 clicks

(Let the video run 3 seconds before you speak.)

So if the answers are already there...

where is the problem?

(pause — let the video breathe)

Here it is.

A new problem is reported today.

Somewhere in that archive, the same problem was already solved.

Maybe two years ago. Maybe by another team, in another building.

► **CLICK**

But nobody remembers it. And nobody can find it.

► **CLICK**

So the team starts again, from zero.

They investigate. They test. They lose weeks.

And the company pays twice for the same answer.

 **The answer already exists. Nobody can find it.**

Slide 7 — What it costs · ~33 s · 3 clicks

Why can they not find it?

Because the only tools are human memory and scrolling.

And a board can hold thousands of cards.

► **CLICK**

So four things happen.

Time is lost. The same problem is solved twice.

Good solutions stay buried.

And nobody sees which problems keep coming back.

► **CLICK**

(slow, clear — your best sentence of the problem part)

The company is not short of knowledge.

It is short of a way to **reach** it.

 **The knowledge is there. The way to reach it is not.**

Slide 8 — Search by meaning · ~46 s · 4 clicks

You may think: just use the search bar.

I tried. It does not work.

Look at these two issues.

Today the operator writes:

"Scratches on machined parts after the tool change."

In the archive we have:

"Surface defects on milled components."

► **CLICK**

Same problem. Not one word in common.

A keyword search finds **nothing**.

► **CLICK**

So the system must compare **meaning**, not words.

We turn each text into a list of numbers.

Texts that mean the same thing end up close to each other.

► **CLICK**

Text becomes meaning. Meaning is compared. We get a ranked list.

That is the whole idea of this project.

 **Same problem, different words. So we compare meaning, not words.**

Slide 9 — The solution · ~2 min 05 · 5 clicks

(The centre of your defense. Do not rush. Let each clip play.)

So this is what I built.

Three steps. **Capture. Find. Reuse.**

Let me show you the real system.

► **CLICK** *(clip 1 — let it play, then speak over it)*

Step one: **Capture.**

Nothing changes for the operator.

He fills the 5W2H exactly like he does today.

No new habit. No new tool to learn.

► **CLICK** *(clip 2)*

Step two: **Find.**

One click.

The system takes this new issue
and compares it with the whole history.

Not by keywords — by meaning, and field by field.

More than two hundred issues, searched in a few milliseconds.

It gives back the closest ones, ranked.

(point at the score bars)

And look — it also shows **why** each one matched.

► **CLICK** *(clip 3)*

Step three: **Reuse.**

This is the part I care about the most.

The system does not only say "this looks similar".

It gives back the root causes that were already found.

The solutions that already worked. And a ready action plan.

(pause)

And every single line says which issue it comes from.

The manager creates these actions on his own board, in one click.

► **CLICK**

Capture. Find. Reuse.

Inside the tool the teams already use every day.



Three steps: capture the problem, find what happened before, reuse what worked.

Slide 10 — Architecture · ~42 s · 4 clicks

Now, how is it built?

(point at the three layers)

Three layers, and one rule: I did not rewrite iObeya.

► **CLICK**

On top, the **interface** — a side panel inside the iObeya issue editor.

It reads the issue already open. The user retypes nothing.

► **CLICK**

In the middle, the **service** — Python and Flask.

This is where the intelligence is. Four endpoints, nothing more.

► **CLICK**

Below, the **storage** — PostgreSQL with pgvector.

The issue and its vectors live in the same row.

► **CLICK**

Two decisions I want to defend. 🕒

A separate service, so it restarts without redeploying iObeya. 🕒

And PostgreSQL instead of a specialised vector database — one query, not two systems. 🕒

🛠️ **Three layers: a panel in iObeya, a Python service, and PostgreSQL with pgvector.**

Slide 11 — Built with · ~26 s · 2 clicks · ✂️ SKIP IN ROUTE A

This is the full stack, layer by layer.

► **CLICK**

I will keep only one sentence.

Flask, and not a heavier framework, because the service is a few endpoints and no interface.

Vue and Docker were not my choice — they are what iObeya already runs.

► **CLICK**

And what is deliberately **absent**:

no GPU, no specialised database, and no paid API for the search to work.

🛠️ **Standard, boring technology on purpose — and no paid API needed.**

Slide 12 — The pipeline · ~40 s · 5 clicks · ✂ SKIP IN ROUTE A

Here is the whole system in four stages.

► **CLICK Ingest** — issues come from the iObeya API and are mapped to the 5W2H.

► **CLICK Embed** — each issue becomes eight lists of numbers:
seven for the fields, one for the whole issue.

► **CLICK Search** — the new issue is compared with everything, exactly,
in twenty-five to forty milliseconds.

► **CLICK Generate** — the issues we found become the summary and the action plan.

► **CLICK**

The next slides open the two that matter.

🔗 **Four stages: ingest, embed, search, generate.**

Slide 13 — Built on 5W2H · ~45 s · 5 clicks

Now, what makes this different from a normal search engine?

A normal engine takes the whole issue and treats it as one block of text.

I do not.

► **CLICK**

An industrial problem has a **structure**. So I compare it field by field.

And every field does not weigh the same.

(point at the bars)

"What happened" carries a third of the score.

"Who saw it" carries almost nothing —

because the same failure seen by two operators is still **one** failure.

► **CLICK**

And two fields are not text at all.

A date is compared as a date. A quantity is compared as a ratio.

► **CLICK ► CLICK**

The result: the system can explain **why** two problems match.

Field by field. Not just a number. 🕒

🔗 **A problem is not a paragraph. It has structure, and I use that structure.**

Slide 14 — RAG · Retrieval · ~52 s · 5 clicks

People call this kind of system **RAG** —
retrieval, then generation. Finding first, writing second.

This slide is the finding. Let me walk one real example.

► **CLICK**

First, the new issue is turned into numbers. One list per field.

► **CLICK**

Then each field is compared with the same field of the old issue.

Here, "where" matches at zero point nine five. "Who" at zero point one two.

► **CLICK**

Each score is multiplied by its weight, and we take the average.

That gives zero point seven two.

► **CLICK**

We mix it with the score of the issue as a whole. 🕒

► **CLICK**

And the final score is **zero point seven one** — "very similar".

Above the limit, so we keep it.

(pause)

These are the issues that the next step is allowed to talk about. And only these.

🛑 **Every field is scored, multiplied by its weight, and averaged into one final score.**

Slide 15 — RAG · Generation · ~40 s · 5 clicks

Second step: generation.

► **CLICK**

The context is only the issues we just found. Nothing else.

► **CLICK**

Then the **code** — not the AI — groups the causes,
merges the repeated actions, and puts them in the right order.

► **CLICK**

Only then, a language model may rewrite that text more nicely.

If there is no model, or it fails, we keep the text written by the code.

► **CLICK**

► **CLICK**

And we measure it.

A sentence quoted from a real issue scores one point zero.

An invented one scores around zero point three five.

So we can **see** whether the summary is based on the evidence.

🛑 **The code builds the content. The model only rewrites it. And we measure it.**

Slide 16 — Grounded AI · ~32 s · 5 clicks

So the honest question is: can we trust it?

► CLICK ► CLICK

The AI is allowed to summarise, to rephrase, to explain.

► CLICK

It is **not** allowed to invent an action.

Not allowed to decide what should be done.

Not allowed to invent evidence.

► CLICK

Every proposed action already exists in a real past issue.

(slow — last sentence)

The AI writes. The expert decides.

 **The AI writes the text. It never invents an action. The expert decides.**

Slide 17 — Results · ~55 s · 4 clicks

Does it actually work? I measured it.

One thousand issues indexed. Two hundred and seven test questions.

Precision around sixty percent. Recall around sixty-five.

► CLICK

But this next number is the one I am proud of.

I took the same questions and wrote them again with **different words**.

Industry synonyms — like a real user would.

(pause, then slowly)

A keyword search lost **fourteen percent** of its quality.

My system lost **zero point two percent**.

► CLICK

Because in real life, people never reuse the exact same words.

That gap is the whole point of this project.

And it is fast: twenty-five to forty milliseconds.

► CLICK

Now I want to be honest with you.

My test data is mostly generated, from a single room.

These numbers show the system **works**.

They do not yet prove how it behaves in a real factory, at full scale.

That is the next step.

 **When the words change, keyword search loses 14%. My system loses 0.2%.**

Slide 18 — Conclusion · ~36 s · 3 clicks

To conclude.

Three things make this system what it is.

It is **semantic** — it searches by meaning, not by words.

It is **domain-aware** — built on the Lean 5W2H, not on generic text.

And it is **grounded** — it returns real solutions, from real issues.

► **CLICK**

What comes next: a real production corpus, validation by Lean experts, and hybrid search to also catch machine codes and part numbers.

► **CLICK**

(stop. look at the jury. say it slowly.)

And this is the sentence I would like you to remember:

Every problem a company solves
should only have to be solved **once**.



Semantic, domain-aware, grounded. Solve it once.

Slide 19 — Thank you · ~14 s

Thank you for your attention.

I will be happy to answer your questions.

(Then stop talking. Stand straight. Wait.)

Q&A — prepare these eleven

Short answers. Never defend — **explain**. If you do not know, say "I did not test that. My honest answer is..." A jury respects that far more than an invented answer.

1. What was the most difficult technical problem you faced?

(This was a slide before. It is now your strongest answer — use it.)

To make the search faster, I added a standard index. Then obvious matches started coming back missing. The index was cutting the data into one hundred parts, and searching only **one** of them. One percent of the history. Nothing crashed. No error, no warning — the answers were simply wrong. I removed the index; it now searches everything, exactly, in 25 to 40 milliseconds. It taught me that a system can be broken and silent at the same time, and that only testing finds that.

2. Why not simply use ChatGPT for everything?

Three reasons. Cost — it must work without a paid API. Privacy — factory data should not leave the company. And most important: a language model **invents**. My action plan must come from real past issues. So the finding does the work, and the AI only writes the text.

3. Your test data is mostly generated. Are the results meaningful?

For the absolute numbers — not yet, and I say so in the report. For the comparison — yes. Both engines were tested on exactly the same data with the same questions. So minus fourteen percent against minus zero point two is a fair comparison.

4. Sixty percent precision is not very high.

It is a strict measure. I count a result as correct only if it shares **both** the same root cause **and** the same solution. A genuinely useful issue that was fixed a different way counts as an error in my numbers. The real usefulness is higher than sixty percent.

5. Why is the threshold at 0.45?

It is a recall-oriented default: I prefer to show a little more, because the interface shows the score and the reason for each match, so the user can judge. I measured the alternative — 0.55 to 0.60 gives more precision and less recall. It is a setting, not code.

6. Does it replace the expert?

No. It never decides. It proposes what worked before and says where it comes from. The manager stays responsible. That was a design rule from the start.

7. What if the problem is completely new?

Then nothing passes the threshold and it returns an empty list. That is deliberate. I prefer to return nothing than to force a bad match and lose the user's trust.

8. Will it still be fast with one million issues?

Today it compares the query with every issue, so the time grows with the data. It is comfortable up to roughly ten to a hundred thousand issues. Above that we bring back an approximate index — but this time we measure its recall first. That is exactly the lesson from question one.

9. Why did you choose the weights by hand instead of learning them?

Because my corpus is partly generated. If I learned the weights from it, I would learn the habits of my data generator, not the judgement of a Lean expert. So the weights come from Lean practice, they are configuration, and the report gives the procedure to re-tune them on real data.

10. Does it work in French, or other languages?

The current model is trained mainly on English. There is a multilingual model with the same size and the same dimension, so switching is a configuration change, not a rewrite. I did not measure it, so I cannot promise the quality yet.

11. Does any factory data leave the company?

No. The default model runs locally, offline, with no API key. That was a requirement from iObeya, and the whole core feature respects it.

One-page cue card

Print this page alone. Put it on the table. Glance, do not read.

#	Slide	The one thing to say
1	Cover	I taught a factory tool to remember .
2	Agenda	Context · problem · solution · difference · results
3	Lean	Gemba = the real place. Go and see.
4	Obeya	iObeya = the digital Toyota "big room". Sanofi, Airbus...
5	Gemba Assistant	5W2H. Solved issues stay on the board, with the solution.
6	Problem 🎬	The answer exists. Nobody can find it. They pay twice.
7	Cost	Not short of knowledge — short of a way to reach it.
8	Meaning	Same problem, no shared words → compare meaning .
9	Solution 🎬🎬🎬	Capture → Find → Reuse. Let the clips play.
10	Architecture	Panel in iObeya · Python service · PostgreSQL. Not a rewrite.
11	Built with ✂️	Boring tech on purpose. No paid API. (<i>skippable</i>)
12	Pipeline ✂️	Ingest · embed · search · generate. (<i>skippable</i>)
13	5W2H	A problem is not a paragraph. What = 1/3. Who ≈ 0.
14	RAG · Retrieval	Score each field × its weight → 0.71, very similar .
15	RAG · Generation	Code builds it. The model only rewrites it.
16	Grounded AI	AI writes. Expert decides. Never invents an action.
17	Results	Words change: keyword -14% , mine -0.2% .
18	Conclusion	Semantic · domain-aware · grounded. Solve it once.
19	Thank you	Thank you. Questions?

Numbers you must not get wrong: 1 000 issues · 207 questions · 60% / 65% ·
-14% vs -0.2% · 25–40 ms · 8 vectors per issue · final score **0.71**.

If you are late: skip 11 and 12. If you are very late, jump to 17.

Good luck. You know this project better than anyone in the room.